Architecture

A React PWA on Firebase Hosting with **no backend of its own.** The browser is the entire application; it talks to Firestore directly, and every constraint in this document follows from that one fact.

REQUEST PATH

Browser · React 19 + Vite, installable PWA

One component file, tab navigation via local state, no router. Service worker cached, auto-update on deploy.

↓ four methods only: get / set / delete / list

Storage interface · `window.storage`

The app never imports Firebase. One file maps this interface onto Firestore, which is what makes every screen testable against a stand-in store with no live project.

↓ attested on every request

App Check · reCAPTCHA Enterprise, enforced

Firestore rejects any request without a valid token. This is what makes a public config object safe to ship in the bundle.

↓

Firestore · one collection, one document per key

Each document holds a JSON string in a `value` field.

EVERYTHING ELSE THE CLIENT TOUCHES

Open-Meteo · forecast

Daily high and low for the session date, plus the hourly UV series averaged across the playing window. No key required.

Google Analytics · behaviour

Screen views per tab and named events per action. Write-only from the browser; reading it back needs a service-account credential, which cannot live in a public bundle.

Formspree · feedback

In-app form to the organizer's inbox. One direction.

Google Calendar · link only

The club schedule cannot be fetched from a browser, so the app links out to it rather than pretending to know what is booked.

Kept out of search engines. A `noindex` directive and a site-wide `robots.txt`. Names and playing times are for the club, not for a crawler.

DELIVERY

Source control	GitHub Desktop, push to `main`
CI	`npm ci && npm run build`
Deploy	Firebase Hosting, static
Round trip	about 45 seconds, no CLI
Version	stamped from `package.json`, shown in the footer

WHY NO BACKEND

This is a volunteer-run club of a few dozen people. A server would mean hosting to maintain, secrets to rotate, and something that can page someone at the weekend.

The cost is paid in four specific places, each documented where it bites: no scheduled jobs, no server-side reads, no credentials, and no cross-client transactions. Nothing here works around those — it accommodates them.

The pairing engine

All pure functions, no side effects, no I/O. **The engine can be exercised with no browser and no database** — which is how every change to it is verified before it ever renders.

HOW A SET IS BUILT

No solver, no global optimum. Each set runs **250 shuffled attempts**, scores each, and keeps the cheapest. Every criterion is converted into one comparable number — cost — so they can be weighed against each other at once.

×	PENALTY
10	Skill imbalance between the two teams
3	Partners who have already played together
2	Spread of competitive ratings on one court
2	Serving imbalance between the two teams
2	Both weak servers on the same team
1	Opponents already faced this session

A 0.5 rating gap costs 5, so skill dominates and everything else breaks ties beneath it. Sit-outs carry their own penalty: anyone benched below the session average costs the round proportionally, and counts carry between sets.

> **Partner variety is session-scoped.** Repeat avoidance works across the sets of one session and starts fresh on the next. Carrying it across weeks is possible now that match history is real, and is not yet built.

WHAT FEEDS IT

FIELD	USED FOR
CTA rating	Skill. The club's own scale, applied to everyone
USTA rating	Skill, but only for players with no CTA rating
Competitive 1–5	Grouping temperaments on a court
Serving 1–5	Team composition, not individual skill
Sex	Mixed and split-by-sex formats
Preferred court	Honoured every set

Nothing else reaches the engine. Handedness, age and the free-text note are stored but never read by it, and match results never move a rating.

> **CTA leads, USTA falls back.** The club rates everyone; USTA covers only those who play league tennis. Rating half the club on an external scale and half on an internal one would mean the two halves were never calibrated against each other.

TWO DECISIONS WORTH KNOWING

Serving is a team term

A strong serve is already part of why someone is rated where they are, so adding it to their rating would count it twice. What a rating cannot express is a property of the *pair*: two weak servers together struggle to hold serve however well they rally. Unrated players are skipped, never defaulted.

Ratings never move on results

At a recreational club a rating is a social fact as much as a measured one. The streak flame and the upset tag are display only; neither reads back into pairing.

FORMATS BELONG TO THE SET

Mixed — mixed doubles, plus a singles court when exactly two would otherwise have nothing to fill. **Split by sex** — women's and men's doubles wherever four of a sex are playing, women on the higher courts. **Singles** — the whole set, two per court.

> **Single-sex courts must fill first.** Courts are filled in list order, so one placed last would find the mixed courts had already taken the women, and would silently degrade to ordinary doubles.

Data model & concurrency

One Firestore collection, `court-call-data`, one document per key. **Every write replaces a whole document** — the single most important thing to understand about this system's failure modes.

DOCUMENTS

KEY	HOLDS	SCOPE
`player-directory`	Every member: name, sex, ratings, handedness, competitive and serving ratings, comments, court preference, active flag	Shared
`player-directory-backup`	Last 10 snapshots, each with timestamp, reason and who took it	Shared
`match-history`	Every logged result, keyed by date, set, court and the players in it	Shared
`admin-registry`	Superuser and Admin PINs, each a SHA-256 hash with its own salt	Shared
`audit-log`	Admin and Superuser actions, newest first, capped at 500	Shared
`weekly-state-v2:…`	One week's sheet: who's playing, session window, set formats, schedule	PER BROWSER
`usage-telemetry:…`	Visits, engagement, tab and action counts, device class	PER BROWSER

Known limitation, accepted. The directory is one document, so two Admins editing different players simultaneously overwrite each other — last write wins, silently. Snapshots make it recoverable; the real fix is one document per player, which is a migration rather than a patch.

WHY SHEETS ARE PER BROWSER

A single shared weekly document means two organizers with the app open silently erase each other's work, because every save rewrites the whole thing from that device's memory.

Each browser mints a random id into `localStorage` and namespaces its own weekly and telemetry documents. Collisions become impossible rather than unlikely, with no locking or merge logic to get wrong.

This is per browser profile, not per device. Two browsers on one laptop are two sheets; an incognito window is a third; clearing site data starts fresh and orphans the old one. The session list is the escape hatch — taking a copy pulls another sheet's *current* state, re-read at that moment rather than from the cached listing.

Without `localStorage` the app falls back to a single shared document rather than breaking.

RETENTION

A weekly sheet is deleted when it has been **idle 8 days AND its date has passed**. Both conditions, always. Anything dated today or later survives at any age, so a week-ahead plan is safe.

- Capped at 3 deletions per app load — a trickle, not a sweep
- Empty sheets are hidden from the list immediately, but still age out normally
- Only weekly sheets are ever deleted. Directory, history, registry and telemetry never are

Cleanup requires a visitor. With no server there are no scheduled jobs, so deletion happens when someone opens the app — one browser tidying up after browsers that never came back. If the club goes quiet for a month, nothing is deleted during it. Firestore TTL would fix this without a backend, but cannot express a two-condition rule.

Access control

Three tiers, no user accounts, no login list to maintain — **a deliberate trade of individual accountability for zero administration** at a volunteer club. The other half of the design is holding as little as possible in the first place.

TIERS

TIER	OPENS	UNLOCKED BY
Open	Marking who's playing, generating and adjusting pairings, logging winners, reading history	Nothing
Admin	The directory, plus editing and deleting history	A named PIN, issued individually
Superuser	Everything above, plus granting and revoking Admin, snapshots and restore, full data export, the activity log and usage	One PIN, behind five taps on the logo

History is readable by anyone on purpose — win/loss records are casual, fun information, unlike the notes kept in the directory.

> **Idle locks.** Admin and history editing lock after 10 minutes idle, Superuser after 5. Timers reset on any interaction, so active work is never interrupted, and a warning appears 60 seconds before with a "still here" button. Locking clears the acting admin's name so the next person's actions can't be misattributed.

PIN STORAGE

Hashed, not encrypted — deliberately

SHA-256 with a unique 16-byte salt per PIN. Encryption would be the wrong tool, not a stronger one: with no server, any key would ship inside the public bundle beside the data it protects. A hash never needs reversing.

The limit that remains

A short PIN has a small keyspace. Hashing keeps plain PINs out of the database; it does not make a short one hard to crack if a hash is ever obtained. The fix is longer PINs, not different hashing.

Registry isolation

Who holds Admin lives in its own document. Admins can already edit the directory, so a flag stored there would sit inside the thing it controls, allowing self-escalation.

STORAGE FAILURES CANNOT DOWNGRADE ACCESS

The storage layer tags every read error as `not-found` or `unavailable`. First-time setup runs only on a confirmed `not-found`.

Every PIN check refuses *any* PIN — including a correct one — when the registry cannot be confirmed either way, and says so. Without that distinction, a browser unable to reach Firestore looks identical to a fresh install, and the app would rebuild default access in memory.

> **A privacy shield blocking the attestation script makes the app unusable, not insecure.** The message names the browser and the setting when it can detect it.

HELD DELIBERATELY NARROW

The directory records **no contact details of any kind** — no phone number, no email, no address.

Without a PIN the app shows a name and nothing else. Ratings sit behind a tap on the info control; competitive, serving and the notes are behind the PIN.

Protecting the irreplaceable data

Pairings regenerate in seconds. **The directory and the match history cannot be reconstructed at all.** Everything on this page exists because of that asymmetry.

IMPORT MERGES, NEVER REPLACES

Rows are matched by name, and **only fields the spreadsheet actually supplies are written**. A blank cell means "nothing to say", never "erase this". A missing column changes nothing.

The alternative — replacing the record with the incoming row — resets every hand-entered value the sheet does not carry. A club roster has names and ratings, so competitive ratings, notes and court preferences would be one import away from silent erasure.

The internal id and the active flag never come from a file: the id must stay stable or match history loses its link to the player, and the sheet's availability column is a stale snapshot.

THREE LAYERS, DELIBERATELY

1 · Review before writing

Choosing a file changes nothing. Every field that would change is listed per player — `CTA 4.5 → 3.0` — with a tick box to exclude anyone whose app values are newer than the sheet. Cancel writes nothing at all.

2 · Snapshot on confirm

Written and verified before the directory changes; if it cannot be saved, the import does not run. Last 10 kept, restorable from the Superuser console. Restoring is itself snapshotted, so it is reversible in both directions.

3 · Off-database export

Importing prompts to download a spreadsheet copy first. The Superuser console offers a full JSON export of every document.

Snapshots share the database they protect. They defend against mistakes, not against losing the project. The downloaded export is the only copy that survives that.

TESTING APPROACH

The app is driven headlessly in jsdom against a mock `window.storage` that reproduces the real error tagging. Tests click real buttons and assert on what reaches storage, rather than calling internals.

- Import merge, per field, from real generated `.xlsx` workbooks
- Blocked-storage behaviour, including default-PIN refusal
- Idle locks, driven by advancing the clock rather than waiting
- Session scoping, 8-day retention boundaries, and copy freshness
- Set formats, single-sex fill order, and rating precedence

Worth the note: this suite has repeatedly caught things a linter could not — a runtime crash from a helper removed alongside dead code, a snapshot written before the user had confirmed anything, and a copy that silently used stale data.

Operating notes & open items

What to check, what is known-limited, and what a future change should not undo without re-reading the reason.

CONSTANTS WORTH KNOWING

SETTING	VALUE
Admin idle lock	`10 min`
History-edit idle lock	`10 min`
Superuser idle lock	`5 min`
Lock warning	`60 s before`
Sheet retention	`8 days idle + date past`
Deletions per load	`3`
Snapshots kept	`10`
Audit log cap	`500`
Telemetry flush	`60 s + on hide`
Engagement idle cutoff	`2 min`
Attempts per set	`250`
Weak serve threshold	`1–2 of 5`
Streak badge threshold	`3 wins`
Max players	`24`

KNOWN AND ACCEPTED

Court numbers are unverified

Assigned from headcount alone. The club calendar cannot be read from a browser, so a printed court number carries no guarantee it is free.

Directory writes are whole-document

Simultaneous Admin edits overwrite each other silently. Snapshots make it recoverable.

Badges depend on complete logging

Streaks are computed from logged results, so an unlogged loss does not break a streak — the flame can overstate, not just lag. The same applies to any future badge derived from results.

Analytics under-counts

Privacy-blocking browsers drop the script entirely. The in-app usage panel counts those people; Google Analytics does not, so the in-app figures are the honest ones.

Console settings live outside the repo

App Check enforcement and the Firestore rules are set in the Firebase console. Nothing in the codebase can confirm them.

IF PICKING THIS UP

Do not simplify import back to a replace

Field-level merge prevents silent data loss, not defensive style.

Do not move the admin registry onto player records

It permits self-escalation. The separation is the control.

Do not let single-sex courts fill last

They silently degrade to ordinary doubles.

Do not treat a failed read as empty

The distinction between `not-found` and `unavailable` is load-bearing.

Watch the pairing quality panel

It records how many sheets went out untouched. A falling figure is the first sign a scoring weight is off.

OPEN, NOT STARTED

- Per-player directory documents
- Repeat-partner avoidance across weeks
- A limitation flag for players returning from injury
- A computed form signal from match history